REPORT

Reviewer report: Delgan/loguru#1515: Evaluate every

rotation condition so a skipped one cannot fire spuriously

Run loguru-1515, tested 2026-11-06 16:01 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

fdb0b0b26f2b258904094b630a23896d273e0575. Report generated 2026-11-06 16:10 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Evaluate every rotation condition so a skipped one cannot fire spuriously

(Delgan/loguru#1515).

• What we observed: not verified: no AI-written probe ran any changed line, so the change itself was not

exercised by a probe; 1 test(s) fail on main and pass on the PR; 148 test(s) ran on main, 148 on the PR; 0 new

test(s) pass, 150 unchanged (3 of them skipped on both sides).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 10.2 GB (of 15.6 GB).

• Setup check: free disk 94.3 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: loguru python setup install instructions https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python documentation install https://github.com/Delgan/loguru site:github.com/Delgan/loguru

• Query: loguru python install requirements https://pypi.org/project/loguru/ https://github.com/Delgan/loguru

site:pypi.org

• Source: loguru/pyproject.toml at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/pyproject.toml

• Source: lager · PyPI — https://pypi.org/project/lager/

• Source: Python Logging (beans-logging) - PyPI — https://pypi.org/project/beans-logging/

• Source: loguru - PyPI — https://pypi.org/project/loguru/

• Source: loguru/CONTRIBUTING.rst at master · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/blob/master/CONTRIBUTING.rst

• Source: GitHub - Delgan/loguru: Python logging made (stupidly) simple · GitHub —

https://github.com/delgan/loguru

• Source: Can‘t Import loguru · Issue #116 · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/issues/116

• Source: Suggestion/request: Documentation/tutorial for beginners · Issue #71 · Delgan/loguru · GitHub —

https://github.com/Delgan/loguru/issues/71

• Setup plan: the same packages on main and pr.

• Setup plan: colorama is noted, installed with the project.

• Setup plan: aiocontextvars is noted, installed with the project.

• Setup plan: win32-setctime is noted, installed with the project.

• Setup plan: install pre-commit==4.6.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install tox==3.27.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest==6.1.2 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-cov==2.12.1 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install pytest-mypy-plugins==4.0.3 ; python_version>='3.12' (listed in the dev extra) on main and pr.

• Setup plan: install freezegun==1.1.0 ; python_version<'3.8' (listed in the dev extra) on main and pr.

• Setup plan: install exceptiongroup==1.1.3 ; python_version>='3.7' and python_version<'3.11' (listed in the dev

extra) on main and pr.

• Setup plan: install mypy==v0.910 ; python_version<'3.6' (listed in the dev extra) on main and pr.

• Setup plan: install Sphinx==8.2.1 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install sphinx-rtd-theme==3.0.2 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install myst-parser==4.0.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install build==1.3.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: install twine==6.2.0 ; python_version>='3.11' (listed in the dev extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.5, <4.0,>=3.5

• Setup plan: Python rule: one Python version is used on both sides: the lowest version that meets the

requirements of both main and the pull request; if none meets both, the check stops as a setup problem and

records both.

• Repository crawl: 9 page(s) crawled, 8 read in full, 1 setup step(s) used first in the sandbox (0 Tavily call(s), 0.0

credit(s)).

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C1 | Evaluate every rotation condition so a skipped one cannot fire spuriously (PR title) | not testable | claims

are read out of the PR description only by model-written probes (--model-provider, see Probes); the tests below

check behavior | pytest-pr-1.log:8 «148 passed, 3 skipped in 26.01s»

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (150 unchanged, 3 runs per side) | pytest-pr-1.log:8 «148 passed, 3 skipped in

26.01s»

Citation check (PASS): 2 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 148 / 148 | | Tests unchanged | 150 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result

in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 1 |

tests.test_filesink_rotation::test_multiple_rotation_conditions_all_evaluated

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

No probe ran for this PR (no fixed probe in probes/<owner>/<repo>/<pr>.py and no model-written or differential

probe), so only the tests were compared.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (32.1 s run).

Coverage gate (python, confidence normal): 5 of 5 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| loguru/_file_sink.py | 5 | 5

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 6c91de2c42d3 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 5 of 5 changed executable line(s) ran (100%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Every claim cites a real log line: 2 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | not verified: no AI-written probe ran any changed line, so the change

itself was not exercised by a probe; 1 test(s) fail on main and pass on the PR; 148 test(s) ran on main, 148 on the

PR; 0 new test(s) pass, 150 unchanged (3 of them skipped on both sides) | Read the setup/build log under

Evidence links; build failures usually need a compiler or a newer nightly.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 5 of 5 changed executable line(s) ran (100%)

| Probes ran on both sides | SKIP | no probe for this PR

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | SKIP | not replayed yet

| Every claim cites a real log line | PASS | 2 citation(s), every one re-read from its log file and matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-222-loguru-live/reports/daily/2026-11-06-222-fdb0b0b-v1/loguru-1515

DECISION=approve (or reject, approve_with_notes, re_run).

How long each step took

| Step | Seconds

| sandbox prep (checkout) | 13.6

| Tavily setup lookup | 10.7

| demo_helper | 0.0
| Python setup | 0.0
| dependency install | 0.0
| model: claims/probe writing | 8.2
| probes | 160.2
| main side install | 71.6
| tests (both sides) | 297.2
| verdict | 0.0
| record | 0.0
| tests (main side) | 87.7
| tests (PR side) | 87.2
| model call 1 (claims) | 8.2
| model call 2 (probes) | 116.9
| model call 3 (probes) | 43.3

Rerun command
make check PR=https://github.com/Delgan/loguru/pull/1515

Evidence links
• Pull request Delgan/loguru#1515
• record.json
• pr.diff
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml

• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• main run 1 result: pytest-main-1.log:8 «147 passed, 3 skipped in 26.03s»
• pr run 1 result: pytest-pr-1.log:8 «148 passed, 3 skipped in 26.01s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-06 16:11 IST